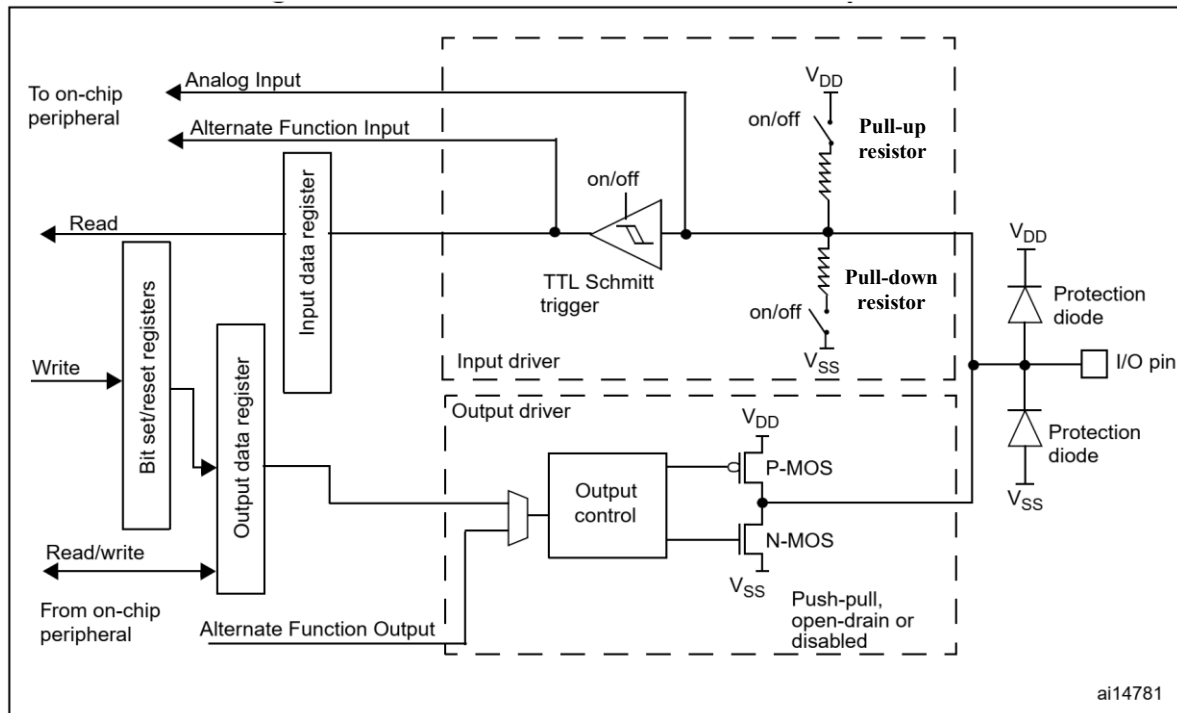


Bài giảng: Cấu hình chân I/O trên vi điều khiển STM32F103

1. Cấu trúc phần cứng chân I/O STM32



Chân I/O trên STM32 được cấu tạo gồm các khối chính:

- Mạch bảo vệ: 2 diode bảo vệ nối đến V_{DD} và V_{SS} , tránh hư hỏng khi có điện áp vượt quá giới hạn.
- Ngõ vào (Input driver):
 - Gồm bộ so sánh TTL Schmitt Trigger để lọc nhiễu và xác định mức logic.
 - Điện trở kéo lên (Pull-up) và kéo xuống (Pull-down):
 - Có thể bật/tắt bằng phần mềm.
 - Chỉ hoạt động khi chân ở chế độ ngõ vào (Input mode).
 - Giúp định mức mặc định cho chân, tránh bị nhiễu khi không có tín hiệu ngoài.
- Ngõ ra (Output driver): gồm transistor P-MOS và N-MOS để điều khiển mức cao/thấp (push-pull, open-drain...).

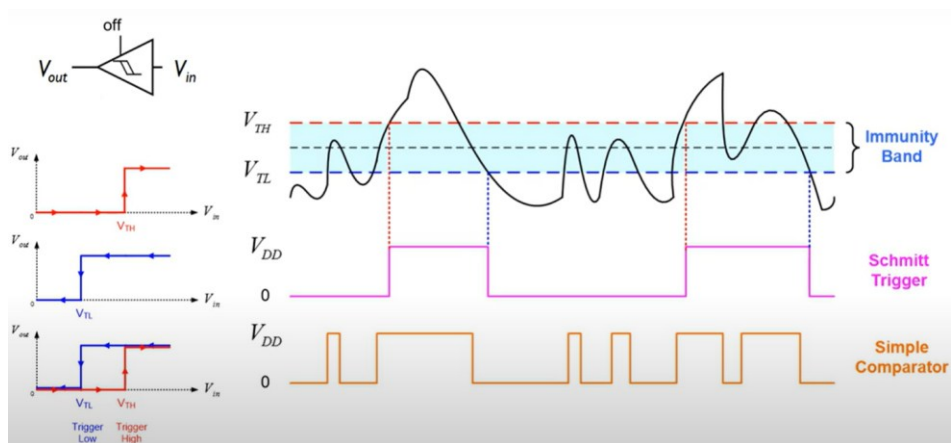
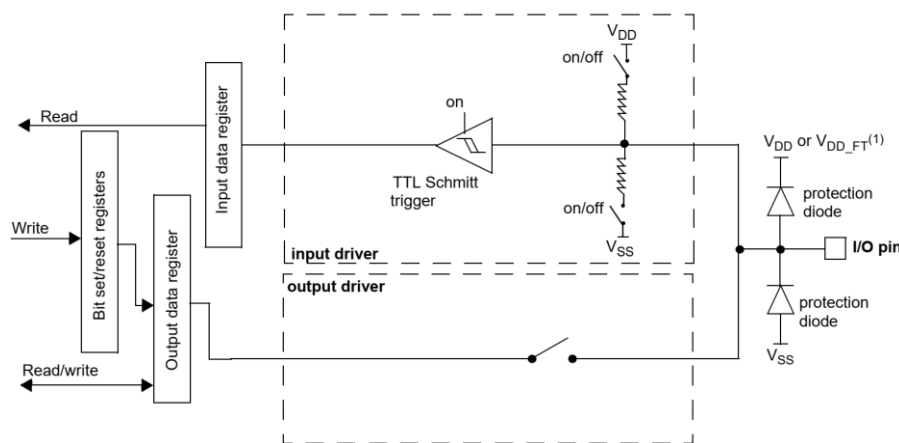
- Mạch chức năng thay thế (Alternate Function): kết nối pin đến ngoại vi khác bên trong chip (UART, SPI, I²C, PWM...).

2. Các chế độ hoạt động của chân STM32

a) Input mode (Chế độ ngõ vào)

1. Input floating:

- Không dùng pull-up hoặc pull-down.
- Pin ở trạng thái " thả nổi" → dễ bị nhiễu nếu không kết nối ra ngoài.
- Dùng khi tín hiệu đã có mạch ngoài xác định mức logic.



2. Input pull-up:

- Kích hoạt điện trở kéo lên nội bộ → pin mặc định ở mức logic 1 khi không kết nối.
- Tránh nhiễu, dùng cho nút nhấn kéo xuống GND.

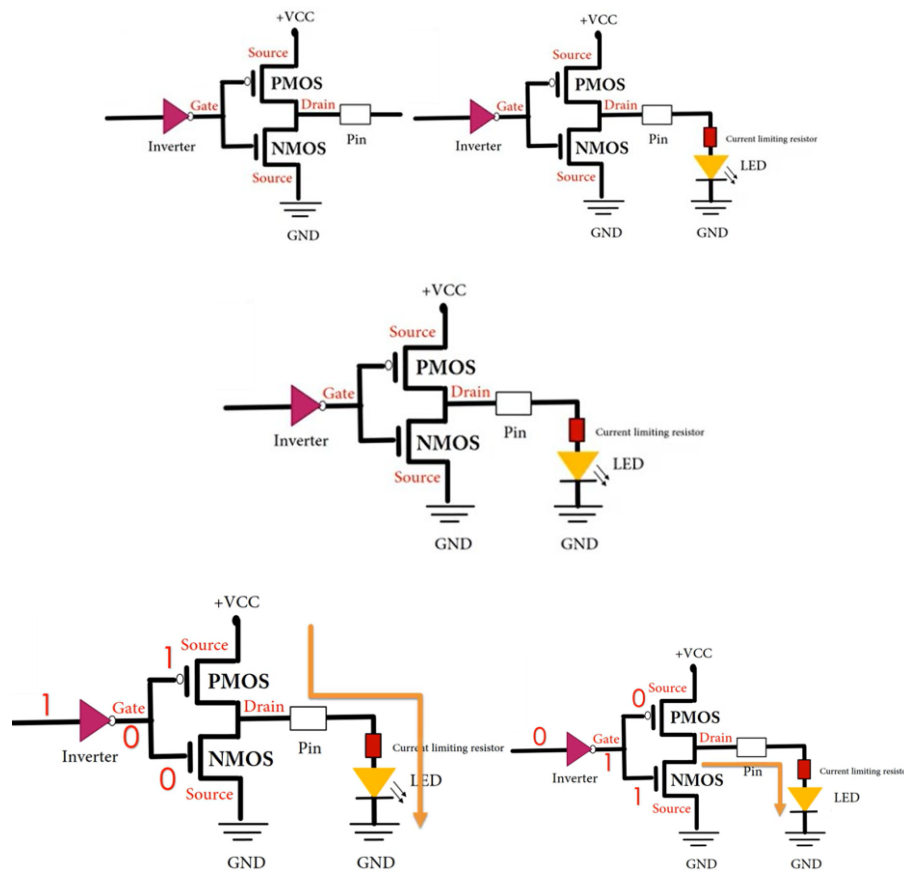
3. Input pull-down:

- Kích hoạt điện trở kéo xuống nội bộ → pin mặc định ở mức logic 0.
- Dùng cho nút nhấn kéo lên VDD.

b) Output mode (Chế độ ngõ ra)

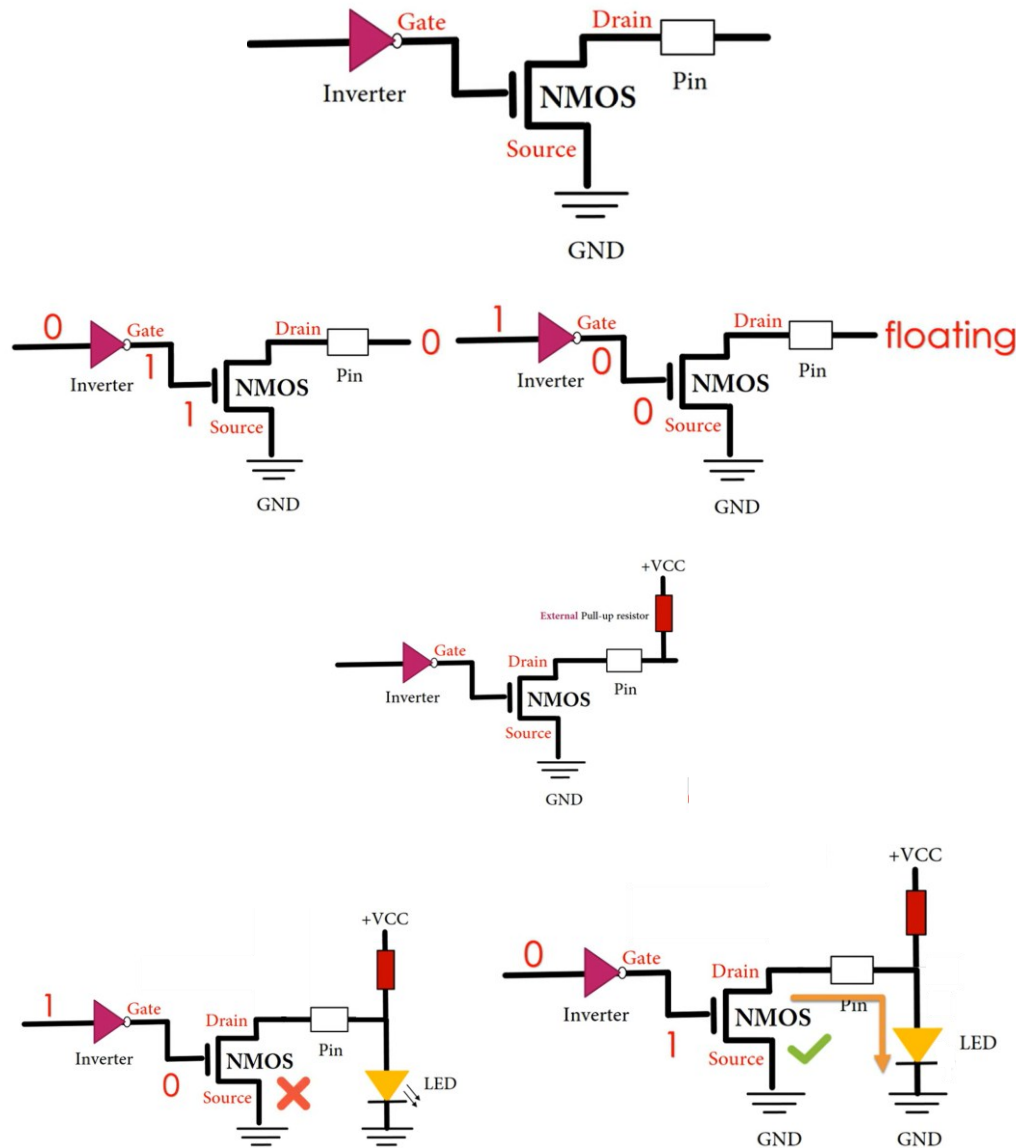
1. Push-pull:

- P-MOS kéo lên VDD, N-MOS kéo xuống GND.
- Cho phép xuất mức logic cao/thấp mạnh, thích hợp điều khiển LED, IC logic.



2. Open-drain:

- Chỉ có N-MOS kéo xuống GND, mức cao phải qua điện trở kéo ngoài hoặc pull-up nội bộ.
- Dùng cho bus I²C, tín hiệu mức cao linh hoạt.



c) Alternate function mode

- Chuyển pin sang chức năng ngoại vi (USART TX/RX, SPI, I²C, PWM...).
- Có thể kết hợp với push-pull hoặc open-drain tùy yêu cầu.

d) Analog mode

- Ngắt bộ so sánh Schmitt Trigger và driver → tránh tiêu thụ dòng.
- Dùng cho ADC hoặc DAC.

3. Minh họa từ sơ đồ

- Đường Input: TTL Schmitt Trigger nhận tín hiệu → ghi vào Input Data Register (IDR).
- Đường Output: Output Control điều khiển P-MOS/N-MOS → xuất ra mức cao/thấp.
- Pull-up / Pull-down: bật/tắt qua thanh ghi cấu hình (GPIOx_CRL/CRH + ODR).
- Alternate Function: kết nối trực tiếp pin đến ngoại vi bên trong chip.

3. Các thanh ghi liên quan đến Cấu hình chân I/O trên vi điều khiển

STM32F103

Thanh ghi	Kích thước	Phạm vi tác động	Chức năng chính
GPIOx_CRL (Port Configuration Low Register)	32 bit	Pin 0 → Pin 7	Cấu hình MODE (tốc độ / input) và CNF (kiểu chân) cho 8 chân thấp
GPIOx_CRH (Port Configuration High Register)	32 bit	Pin 8 → Pin 15	Cấu hình MODE và CNF cho 8 chân cao
GPIOx_IDR (Input Data Register)	16 bit	Toàn bộ 16 pin	Đọc trạng thái mức logic của các chân (0 = thấp, 1 = cao)
GPIOx_ODR (Output Data Register)	16 bit	Toàn bộ 16 pin	Ghi mức logic cho chân output; đồng thời chọn pull-up/pull-down khi ở chế độ input
GPIOx_BSRR (Bit Set/Reset Register)	32 bit	Toàn bộ 16 pin	16 bit thấp: Set mức cao; 16 bit cao: Reset mức thấp (không ảnh hưởng bit khác)
GPIOx_BRR (Bit Reset Register)	16 bit	Toàn bộ 16 pin	Reset mức thấp cho chân được chọn

Thanh ghi	Kích thước	Phạm vi tác động	Chức năng chính
GPIOx_LCKR (Lock Register)	16 bit	Toàn bộ 16 pin	Khóa cấu hình chân (MODE/CNF) để tránh thay đổi ngoài ý muốn
RCC_APB2ENR (APB2 Peripheral Clock Enable Register)	32 bit	Bật clock cho port	Phải bật clock trước khi cấu hình GPIO (bit 2 cho GPIOA, bit 3 cho GPIOB, ...)

3.1. Thanh ghi RCC->APB2ENR

RCC->APB2ENR (APB2 Peripheral Clock Enable Register) là thanh ghi dùng để bật/tắt xung clock cho các ngoại vi nằm trên bus APB2 của STM32F103.

Mỗi bit trong thanh ghi tương ứng với một ngoại vi. Khi bit được set = 1, clock của ngoại vi đó được bật.

Bit	Tên bit	Mô tả	0 = Tắt / 1 = Bật
31–16	Reserved	Không sử dụng.	—
15	Reserved	Không sử dụng.	—
14	USART1EN	Bật clock cho USART1.	0 / 1
13	Reserved	Không sử dụng.	—
12	SPI1EN	Bật clock cho SPI1.	0 / 1
11	TIM1EN	Bật clock cho Timer 1.	0 / 1
10	ADC2EN	Bật clock cho ADC2.	0 / 1
9	ADC1EN	Bật clock cho ADC1.	0 / 1
8	Reserved	Không sử dụng.	—
7	Reserved	Không sử dụng.	—
6	IOPEEN	Bật clock cho GPIO Port E.	0 / 1
5	IOPDEN	Bật clock cho GPIO Port D.	0 / 1
4	IOPCEN	Bật clock cho GPIO Port C.	0 / 1
3	IOPBEN	Bật clock cho GPIO Port B.	0 / 1
2	IOPAEN	Bật clock cho GPIO Port A.	0 / 1
1	Reserved	Không sử dụng.	—
0	AFIOEN	Bật clock cho Alternate Function I/O.	0 / 1

Ví dụ: Lệnh `RCC->APB2ENR |= 0x10;` sẽ bật bit 4 (IOPCEN), tức bật clock cho GPIO Port C.

3.2. Thanh ghi GPIOx_CRL và GPIOx_CRH (x=A..G)

✓ Mỗi port (GPIOA, GPIOB, GPIOC, ...) có hai thanh ghi cấu hình:

- **GPIOx_CRL**: cấu hình cho các chân số 0 → 7.
- **GPIOx_CRH**: cấu hình cho các chân số 8 → 15.
- Mỗi thanh ghi có kích thước **32 bit**.
- Mỗi chân cần **4 bit** để định nghĩa chế độ.

Vậy mỗi chân được cấu hình bằng 4 bit: MODE[1:0] và CNF[1:0].

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]				MODE7[1:0]				CNF6[1:0]				MODE6[1:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]				MODE3[1:0]				CNF2[1:0]				MODE2[1:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF15[1:0]				MODE15[1:0]				CNF14[1:0]				MODE14[1:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF11[1:0]				MODE11[1:0]				CNF10[1:0]				MODE10[1:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

3.2.1. MODE[1:0] – Chọn chế độ/Tốc độ Output

MODE[1:0]	Chức năng
00	Input mode (reset state)
01	Output mode, max speed 10 MHz
10	Output mode, max speed 2 MHz
11	Output mode, max speed 50 MHz

3.2.2. CNF[1:0] – Chế độ Output (khi MODE[1:0]#00)

CNF[1:0]	Chức năng
00	General purpose output push-pull
01	General purpose output open-drain
10	Alternate function output push-pull
11	Alternate function output open-drain

3.2.3. CNF[1:0] – Chế độ Input (khi MODE[1:0]=00)

CNF[1:0]	Chức năng
00	Analog mode
01	Floating input (reset state)
10	Input with pull-up / pull-down

3.2.4 Cách tính vị trí bit

Cách xác định vị trí bit khi cấu hình chân GPIO trên STM32F103:

1. Nếu số pin nhỏ hơn 8 (từ 0 đến 7):

- Các bit cấu hình nằm trong thanh ghi GPIOx_CRL.
- Vị trí bit bắt đầu = số pin $\times 4$.
- Ví dụ: Pin 5 $\rightarrow 5 \times 4 = 20 \rightarrow$ chiếm các bit [23:20] của GPIOx_CRL.
- **Code: Cấu hình PA5 là output push-pull 2 MHz:**
// Xóa cấu hình cũ (4-bit của pin 5)
GPIOA->CRL &= ~(0xF << (4 * 5));

// CNF[1:0] = 00 (push-pull), MODE[1:0] = 10 (2 MHz) $\rightarrow 0b0010 = 0x2$
GPIOA->CRL |= (0x2 << (4 * 5));

2. Nếu số pin từ 8 trở lên (từ 8 đến 15):

- Các bit cấu hình nằm trong thanh ghi GPIOx_CRH.
- Vị trí bit bắt đầu = (số pin - 8) $\times 4$.
- Ví dụ: Pin 12 $\rightarrow (12 - 8) \times 4 = 16 \rightarrow$ chiếm các bit [19:16] của GPIOx_CRH.
- **Code: Cấu hình PB12 là input pull-up:**
// Xóa cấu hình cũ (4-bit của pin 12)
GPIOB->CRH &= ~(0xF << (4 * (12 - 8)));

// CNF[1:0] = 10 (pull-up/pull-down); MODE[1:0] = 00 (input)


```
// → 0b1000 = 0x8
GPIOB->CRH |= (0x8 << (4 * (12 - 8)));

// Chọn pull-up bằng cách set ODR = 1
GPIOB->ODR |= (1 << 12);
```

3.3. Thanh ghi GPIOx_ODR

GPIOx_ODR (Output Data Register) lưu và điều khiển giá trị logic xuất ra ở các chân GPIO khi ở chế độ Output.

- Bit y (ODRy) = 1: xuất mức cao (VDD) tại chân Px_y.
- Bit y (ODRy) = 0: xuất mức thấp (0V) tại chân Px_y.

Có thể đọc ODR để biết trạng thái hiện tại.

3.4. Thanh ghi GPIOx_IDR – Input Data Register

- **Địa chỉ:** Thuộc mỗi cổng GPIO (GPIOA, GPIOB, GPIOC, ...)
- **Chức năng:** Lưu giá trị logic đang được đọc tại các chân GPIO.
- **Chỉ đọc** (Read-only).

3.5 GPIOx_BSRR – Bit Set/Reset Register

- **Chức năng:** Dùng để **đặt** (set) hoặc **xóa** (reset) một bit của chân I/O mà không ảnh hưởng tới các chân khác.
- **Cấu trúc:** 32 bit
 - [15:0]: Bit Set – ghi 1 vào vị trí này để **đưa chân tương ứng lên mức 1**.
 - [31:16]: Bit Reset – ghi 1 vào vị trí này để **đưa chân tương ứng xuống mức 0**.
- **Ưu điểm:** Thao tác **an toàn, tránh xung đột** khi nhiều tiến trình cùng truy cập ODR.

Ví dụ: Set pin PA5 lên mức 1

```
GPIOA->BSRR = (1 << 5); // Set PA5 = 1
Reset pin PA5 xuống mức 0
GPIOA->BSRR = (1 << (5 + 16)); // Reset PA5 = 0
```

3.6. GPIOx_BRR – Bit Reset Register

- **Chức năng:** Chỉ dùng để **xóa** (reset) bit chân I/O về 0.
- **Cấu trúc:** 16 bit (chỉ có vùng reset, tương đương [31:16] của BSRR).
- **Ưu điểm:** Viết code gọn hơn khi chỉ cần reset mà không set.

Ví dụ: Reset pin PA5

```
GPIOA->BRR = (1 << 5); // PA5 = 0
```

3.7. GPIOx_LCKR – Lock Register

- **Chức năng:** Khóa cấu hình chân I/O để ngăn thay đổi ngoài ý muốn (phần CRL/CRH).
- Khi đã khóa, chỉ reset chip mới thay đổi được cấu hình chân.
- **Cấu trúc:** 16 bit chân + 1 bit LOCK (bit 16).
- **Trình tự khóa:**
 1. Ghi giá trị cần khóa vào bit [15:0].
 2. Ghi 1 vào bit 16 (LOCK).
 3. Ghi 0 vào bit 16.
 4. Ghi 1 vào bit 16 lần nữa.
 5. Đọc lại bit 16, nếu =1 nghĩa là khóa thành công.

Ví dụ:

```
// Khóa cấu hình PA5
GPIOA->LCKR = (1 << 5);    // Bước 1
GPIOA->LCKR |= (1 << 16);    // Bước 2
GPIOA->LCKR &= ~(1 << 16);   // Bước 3
GPIOA->LCKR |= (1 << 16);    // Bước 4
volatile uint32_t tmp = GPIOA->LCKR; // Bước 5 (đọc lại)
```

3.8. Bài tập:

3.8.1 Bài tập 1: Cấu hình PC13 làm Output Push-Pull

1. Bật clock cho Port C: `RCC->APB2ENR |= (1 << 4);`
2. PC13 thuộc CRH, cấu hình `MODE13 = 10` (Output 2 MHz), `CNF13 = 00` (Push-pull).
3. Ghi vào `GPIOC->CRH` để thiết lập.

3.8.2 Bài tập 2: Blink LED trên chân PC13

```
#include "stm32f10x.h"

int main(void) {
    RCC->APB2ENR |= (1 << 4); // Bật clock Port C
    GPIOC->CRH &= ~(0xF << 20); // Xóa cấu hình cũ của PC13
    GPIOC->CRH |= (0x2 << 20); // MODE13 = 10 (Output 2 MHz)
    GPIOC->CRH |= (0x0 << 22); // CNF13 = 00 (Push-pull)

    while(1) {
        GPIOC->ODR ^= (1 << 13); // Đảo trạng thái PC13
        for (volatile int i = 0; i < 500000; i++); // Delay
    }
}
```

3.8.3. Bài tập 3: PB1 input – PC13 output, trong đó:

- **PB1: nút nhấn active high (kéo lên mức cao khi nhấn).**
- **PC13: LED, đổi trạng thái khi nhấn và thả nút xong.**
- **Có chống dội phím (debounce).**

```
#include "stm32f10x.h"

// Delay tương đối đơn giản (dùng vòng lặp)
void delay_ms(uint32_t ms) {
    uint32_t i;
    for (; ms > 0; ms--) {
        for (i = 0; i < 8000; i++); // Tùy tần số clock, 8MHz ~ 1ms
    }
}

int main(void) {
    // 1. Bật clock cho GPIOB và GPIOC
    RCC->APB2ENR |= (1 << 3); // IOPBEN = 1 (Bật clock cho Port B)
    RCC->APB2ENR |= (1 << 4); // IOPCEN = 1 (Bật clock cho Port C)

    // 2. Cấu hình PB1 là input floating
    GPIOB->CRL &= ~(0xF << (1 * 4)); // Xóa cấu hình cũ cho pin PB1
    GPIOB->CRL |= (0x4 << (1 * 4)); // MODE = 00, CNF = 01 → Input floating

    // 3. Cấu hình PC13 là output push-pull, tốc độ 2MHz
    GPIOC->CRH &= ~(0xF << ((13 - 8) * 4)); // Xóa cấu hình cũ cho pin PC13
    GPIOC->CRH |= (0x2 << ((13 - 8) * 4)); // MODE = 10 (2MHz), CNF = 00 (Push-pull)

    while (1) {
        // Nếu nút PB1 được nhấn (mức cao)
        if (GPIOB->IDR & (1 << 1)) {
            delay_ms(20); // Chống dội phím
            // Kiểm tra lại xem nút vẫn được nhấn
            if (GPIOB->IDR & (1 << 1)) {
                // Đổi trạng thái LED
                GPIOC->ODR ^= (1 << 13);
                // Chờ đến khi nút thả ra (mức thấp)
                while (GPIOB->IDR & (1 << 1));
            }
        }
    }
}
```